Paseo 入门

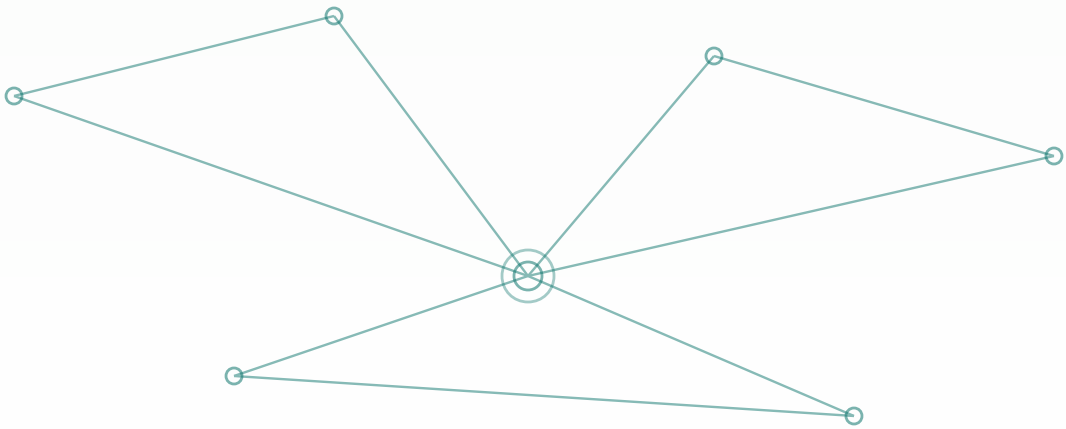
Paseo v0.11.2（2026-10-09）

v0.1.0

为 已在使用至少一个 AI 编程 agent 的开发者 用户创建

鸟窝

2026-10-11



目录

Part 1 起步

第一章 为什么是 Paseo：一个 agent 不够用了

1.1 我的时间线：从单 agent 到多 agent

1.2 对比：直接用 agent CLI，还是用 Paseo

1.3 核心洞察：Paseo 是管理层，不是执行者

1.4 适合谁，不适合谁

第二章 安装和第一次跑通

2.1 你需要什么

2.2 我们最终要做成什么

2.3 三条安装路

2.4 启动 daemon

2.5 跑第一个 agent

2.6 踩坑记录

2.7 回顾

第三章 第一个真实项目：并行跑两个 agent

3.1 项目需求

3.2 worktree 原理一句话

3.3 并行实现

3.4 审批：agent 改文件前要问一下

3.5 结果：两个 agent 各交各的作业

3.6 审查、合并与冲突

3.7 清理：归档工作区

3.8 回顾

Part 2 核心能力

第四章 Workspaces：Paseo 的工作模型

4.1 围绕工作区，不是围绕聊天

4.2 项目 → 工作区 → 会话

4.3 Local vs Worktree：两种隔离

4.4 生命周期：工作区不是永久存在的

4.5 paseo.json：工作区的自动化配置

4.6 实战：把现有仓库变成工作区

第五章 Worktree 并行开发：杀手锏

5.1 原理：独立目录 + 独立分支

5.2 三种模式

5.3 服务与端口：让 agent 跑起开发服务器

5.4 实战：一个仓库，多个任务

5.5 冲突与清理

5.6 回顾

第六章 多 Provider 编排

6.1 Provider 抽象：你装 CLI，Paseo 运行它

6.2 原生 6 个 + ACP 36 个

6.3 模型 ID：provider/model

6.4 Orchestration：让 agent 委派 agent

6.5 Agent profiles：把常用设置存起来

6.6 实战：Claude 规划 + Codex 实现 + 独立审查

6.7 回顾

Part 3 进阶实战

第七章 跨设备：手机到云端

7.1 连接模型：daemon / relay / 客户端

7.2 手机配对与加密

7.3 Web UI 与远程 daemon

7.4 Docker 自托管

7.5 回顾

第八章 自动化：Schedules、MCP 与 Browser

8.1 Schedules：定时任务

8.2 Heartbeats：让 agent 续命

8.3 MCP：Paseo 工具目录

8.4 Browser：agent 自己开浏览器验证

8.5 回顾

第九章 扩展：Skills、插件与 SDK

9.1 Skills：让 agent 编排 agent

9.2 插件：改 Paseo 本身

9.3 TypeScript SDK：把 Paseo 嵌进你的程序

9.4 回顾

第十章 心态转变：一个人，一群 agent

10.1 时间线：一年，2 万 star

10.2 我的工作流

10.3 从"写代码的人"到"管 agent 的人"

10.4 最后一句话

附录

附录 A CLI 命令速查表

附录 B 常见问题排查

附录 C 版本时间线

A Complete Guide to Orchestrating Multiple Coding Agents



Part 1: 起步

从零到一。读者读完能跑通第一个并行 agent 项目。

第一章 为什么是 Paseo：一个 agent 不够用了

先给结论：Paseo 解决的问题，不是"没有 AI 编程工具"。而是"AI 编程工具太多，没人管"。

我用 Claude Code 写了几个月代码。最大的感受：一个 agent 很强，但它是单线程的。它不能一边改登录模块，一边并行修性能问题。它也不能同时让 Claude 做规划、让 Codex 做实现。

2025 年 10 月 13 日，一个叫 Paseo 的项目在 GitHub 上创建。2026 年 2 月 11 日，它发布第一个公开版本 v0.1.2。2026 年 10 月 11 日我查数据时，它已经攒了 20,414 个 star。

一年，从零到 2 万 star。这个数字背后的信号就一个：多 agent 编排，是刚需。

1.1 我的时间线：从单 agent 到多 agent

我的路径是这样过来的。

第一年，我用 Copilot 补全代码。它像输入法。我在写，它在猜。

第二年，我切到 Claude Code。它像结对工程师。我给需求，它改代码。这个阶段很爽，但问题来了。一个任务没跑完，另一个任务得排队。

第三年，我同时在桌面开两个 Claude Code 窗口。一个做前端，一个做后端。痛苦的是它们互相不知道对方在干什么。改同一个文件，覆盖彼此的代码。合并冲突，我要手工解决。

后来我发现了 Paseo。它的做法是：每个任务一个独立工作区。每个工作区，一个独立 git 分支。agent 并行跑，谁也不踩谁。

> **鸟窝的经验**：单 agent 的瓶颈，不是能力，是并发。
>
> 2026 年 10 月，我在自己机器上装了 Paseo v0.11.2。第一次跑任务：claude-opus-5-5，4 秒完成。第二次换 codex，6 秒完成。同一个界面，两个 provider，不需要切换任何配置。这个体验让我确定：这就是管理多 agent 的方式。

1.2 对比：直接用 agent CLI，还是用 Paseo

维度	直接用 agent CLI	用 Paseo	鸟窝的结论
同时跑几个 agent	1 个	多个并行	Paseo 赢
多 provider 切换	记多套命令	统一界面	Paseo 赢
任务目录隔离	手动建 git 分支	worktree 自动隔离	Paseo 赢
手机上查看进度	不行	iOS / Android	Paseo 赢
定时跑任务	自己写 cron 脚本	schedules 内置	Paseo 赢
学习成本	零	半天	各有取舍

> **重点看**：第一行"同时跑几个 agent"。
>
> 这是 Paseo 存在的全部理由。其余所有功能，都是围绕"并行"长出来的。

1.3 核心洞察：Paseo 是管理层，不是执行者

这里要讲清楚一个关键点：Paseo 自己不写代码。

它不内置任何 agent。它管的是"你机器上已经装好的 agent CLI"。

Claude Code 是 Claude Code，Codex 是 Codex。Paseo 是它们的管理层。它的职责：启动 agent、分配任务、隔离文件、汇总输出。

用一个类比级联说清楚：

Copilot 是你的输入法。你写，它补全。这是被动辅助。

Claude Code 是你的结对工程师。你给需求，它改代码。这是主动协作。

Paseo 是你的工程经理。它不写代码，它安排谁在什么时候写什么。这是独立调度。

递进逻辑很清晰：被动辅助 → 主动协作 → 独立调度。Paseo 站在最上层。

> **核心建议**：先想清楚角色，再动手装。
>
> 你是给指令的人，不是写代码的人。这句话对 Paseo 同样成立。你是调度的人，不是被 agent 调度的人。

1.4 适合谁，不适合谁

先说适合谁。

第一，你已经在用至少一个编码 agent。Claude Code、Codex、OpenCode、Pi，都算。Paseo 不会替你做这个选择。

第二，你手上同时有几个任务。一个在改 bug，一个在做功能，一个在写测试。单线程排不过来。

第三，你想按任务挑模型。登录页用便宜的，架构设计用贵的。一个界面切换。

再说谁不适合。

第一，你只用补全级别的 AI。Copilot 就够，不用上 Paseo。

第二，你的项目不需要并行。个人小项目，一个人一个 agent 慢慢跑，Paseo 帮不上忙。

第三，你反感"机器上跑后台服务"。Paseo 的 daemon 常驻，会一直占用一个端口。

> **鸟窝的经验**：Paseo 是"工程化"的工具，不是"玩具"。
>
> 它的每一项能力，都在解决真实工程里的并发问题：并行任务、文件隔离、多模型切换、定时执行。如果你当前的工作流里没有这些痛点，装了也吃灰。有了痛点，它一天就能值回票价。

上面讲的是"是什么"。下一章讲"怎么做"——把它装起来，跑通第一个 agent。

这一章是 Paseo 的定位。安装细节，下一章详细讲。

第二章 安装和第一次跑通

这一章，把 Paseo 装到你的机器上。跑通第一个 agent。

我实测的路径：npm 装 CLI，启动 daemon，跑一个真实任务。总用时不到 10 分钟。

2.1 你需要什么

先清点前置条件。

第一，一个已安装、已认证的 agent CLI。Claude Code、Codex、OpenCode、Pi、Copilot，至少一个。Paseo 不替你装，也不替你登录。

第二，GitHub CLI（`gh`）。需要安装并认证。Paseo 用它做 PR 相关的工作区和部分编排功能。

第三，Node.js。CLI 是 npm 包，需要 Node 环境。

> ⚠ **注意**：gh 不是可选项。
>
> Paseo 的 worktree 和编排功能依赖 gh。没装的话，PR 相关的功能会报错。装法：`brew install gh`，然后 `gh auth login`。

2.2 我们最终要做成什么

这一章结束，你能做到三件事：

第一，机器上跑着一个 Paseo daemon。它是常驻服务，地址 `127.0.0.1:6767`。

第二，你能用 `paseo provider ls` 看到所有可用的 agent 提供方。

第三，你能用一句话跑起一个真实 agent，并看到它的回复。

> 💡 **鸟窝的经验**：装完先别急着玩高级功能。
>
> 我 2026 年 10 月装的时候，第一件事是确认 daemon 活着、provider 可见。这两件事通了，后面全是水到渠成。

2.3 三条安装路

Paseo 有三种装法。按场景选。

第一条：桌面 app。 最省事。去 paseo.sh/download 下载。打开即装好，daemon 自动启动。适合 Mac / Windows / Linux 桌面用户。

第二条：CLI。 适合服务器、远程机器、喜欢终端的人。我在本机用的就是这条。

```
# 这行是关键：全局安装 Paseo CLI
npm install -g @getpaseo/cli
```

预期结果：命令行能认出 `paseo` 命令。

```
paseo --version
# 0.11.2
```

第三条：Docker。 适合跑在服务器上，全家桶式部署。

```
docker run -d --name paseo \
  -p 6767:6767 \
  -e PASEO_PASSWORD=change-me \
  -v "$PWD/paseo-home:/home/paseo" \
  -v "$PWD:/workspace" \
  ghcr.io/getpaseo/paseo:latest
```

装完打开 `http://localhost:6767`。

> ⚠ **注意**：Docker 镜像不内置 agent CLI。
>
> 我这次没跑 Docker——本机没装。命令来自官方文档。镜像只带 daemon 和 Web 界面，agent CLI 要自己在镜像里装。记住这一点，不然镜像起来发现没有 agent，别懵。

2.4 启动 daemon

CLI 装好，启动 daemon。

```
paseo daemon start
```

预期结果：daemon 在 `127.0.0.1:6767` 上监听。首次启动会问你是否启用 relay 和打印配对二维码。这个后面 第七章 讲，这里先选"本地模式"。

看状态：

```
paseo status
```

预期结果：

```
home: /Users/<你>/.paseo
pid: <进程号>
listen: 127.0.0.1:6767
daemonVersion: 0.11.2
```

注意几个细节。

`home` 指向 `~/.paseo`。Paseo 的配置和本地状态都放这里。环境变量 `PASEO_HOME` 可以改它。

`listen` 是 daemon 的监听地址。默认 `127.0.0.1:6767`。

> ⚠ **注意**：daemon 是常驻进程。
>
> 它会一直跑在后台，占着 6767 端口。不想要了，`paseo daemon stop`。

2.5 跑第一个 agent

daemon 活了，看 provider 列表。

```
paseo provider ls
```

预期结果：一张表，列出可用的 agent 提供方和状态。

我机器上装过 Claude Code、Codex、OpenCode、Pi，所以看到四个 available。没装的就是 unavailable。

现在跑第一个任务。

```
# 这行是关键：--provider 指定用哪个 agent
paseo run --provider claude --wait-timeout 90s --json "Reply with exactly one line: pas
```

预期结果：一个 JSON，`"status": "completed"`。

我这次实际跑的数据：

```
Provider: claude
Model:    claude-opus-5-5
耗时：    4 秒
回复：    paseo-smoke-ok
```

用 `paseo inspect <id>` 可以看细节。模型、模式、token 用量，全在里面。

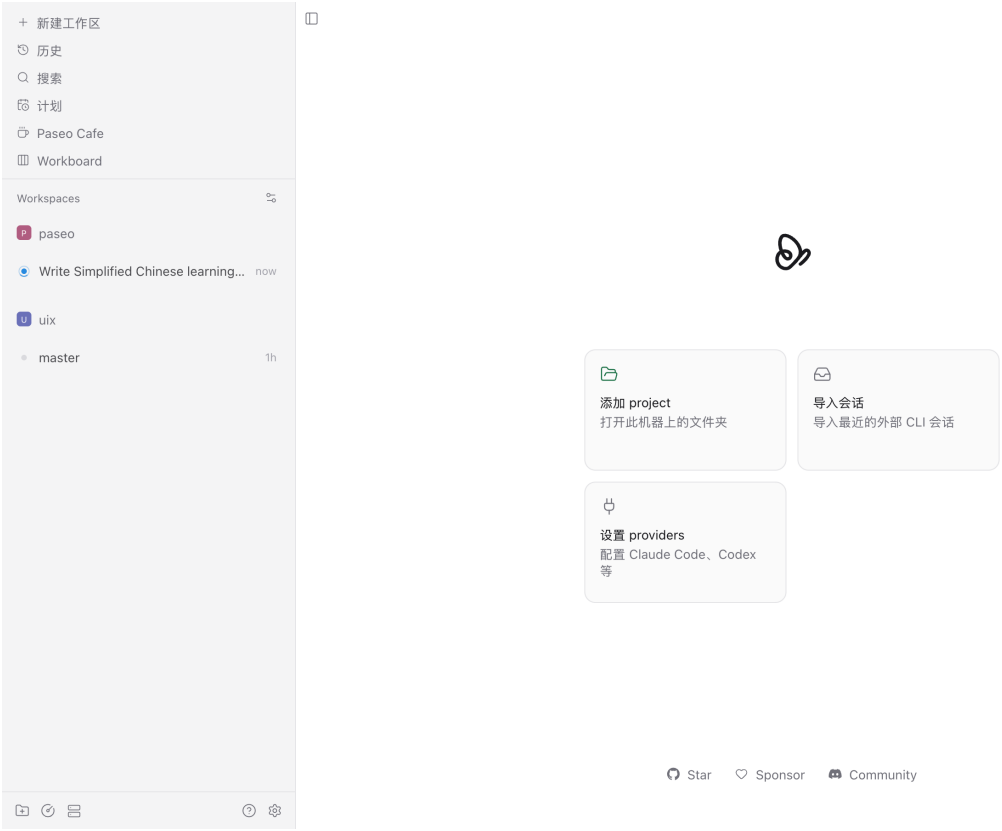
看 agent 列表：

```
paseo ls
```

看实时输出：

```
paseo attach <id>
```

看长什么样。这是我机器上真实打开的 Paseo 界面（Web UI 默认关闭，开启方法在 第 7.3 节）。左侧是工作区列表，右侧空状态给了三个入口：添加项目、导入会话、设置 providers。



图：*Paseo Web UI* 主界面。左侧工作区列表，右侧是空状态操作卡片。

2.6 踩坑记录

装的过程中，两个坑最常见。

坑一：provider 显示 unavailable。 原因几乎都是 agent CLI 没装或没登录。解决：先确认 CLI 能单独跑通，再回来看 Paseo。`paseo provider diagnostic <provider>` 可以看详细诊断。

坑二：`paseo` 命令找不到。 npm 全局安装的 bin 目录没在 PATH 里。解决：确认 npm 全局 bin 目录，加进 PATH。

> **鸟窝的经验**：验证安装，用一个最小的任务。
>
> 不要一上来就让 agent 重构整个项目。先让它回一行字。10 秒出结果。通了，再上真实任务。

2.7 回顾

10 分钟，完成了三件事：CLI 装好，daemon 跑起来，第一个 agent 通了。

> **核心建议**：记住三个命令。
>
> `paseo daemon start` 起服务，`paseo provider ls` 看可用 agent，`paseo run` 跑任务。这三个命令，覆盖 80% 的日常使用。

装完了，daemon 活着，第一个 agent 也通了。下一章，做第一个真实项目——并行跑两个 agent。

第三章 第一个真实项目：并行跑两个 agent

这一章，用真实案例演示 Paseo 的核心用法。2026 年 10 月，我完整跑了一遍。两个 agent 同时开工，互不干扰。

我造了一个迷你项目来演示。一个 Python 计算器 CLI，初始只有加法和减法。两个 agent 并行加功能：一个加乘法，一个加除法。

这个案例我完整跑了一遍。用的工具：Paseo CLI v0.11.2，本地 daemon，codex provider。全程真实记录。

3.1 项目需求

需求很简单。

项目：`calc.py`，一个命令行计算器。初始代码只有 `add` 和 `sub` 两个命令。

任务一：加 `mul`（乘法）命令。实现后自己测一遍，汇报结果。

任务二：加 `div`（除法）命令。实现后自己测一遍，汇报结果。

两个任务改的是同一个文件。这正是 Paseo 要解决的场景。

3.2 worktree 原理一句话

先讲原理，一句话。

worktree 是 git 的一个功能。同一个仓库，可以有好几个独立的工作目录。各占一个分支。

Paseo 把它包装成"工作区隔离"。每个任务一个独立目录、一个独立分支。agent 改文件，各改各的，互相看不见。

我的两个 agent，一个目录叫 `feature-mul`，分支 `feature/mul`。另一个叫 `feature-div`，分支 `feature/div`。它们同时开工，谁也不踩谁。

3.3 并行实现

关键命令是 `paseo run` 加一组 worktree 参数。

```
# 任务一：加乘法，开一个 worktree 工作区
paseo run --new-workspace worktree \
  --worktree-mode branch-off \
  --new-branch feature/mul \
  --worktree-slug feature-mul \
  --base main \
  --provider codex \
  "Add a multiply (mul) command to calc.py. Implement it, run a quick test, and report
```

```
# 任务二：加除法，开另一个 worktree 工作区
paseo run --new-workspace worktree \
  --worktree-mode branch-off \
  --new-branch feature/div \
  --worktree-slug feature-div \
  --base main \
  --provider codex \
  "Add a divide (div) command to calc.py. Implement it, run a quick test, and report th
```

两个命令几乎同时发出。预期结果：

第一，两个 workspace 被创建。

```
Created workspace wks_1ff7562651a0c716 - feature/mul (feature/mul)
Created workspace wks_f9e6cd4293ffb253 - feature/div (feature/div)
```

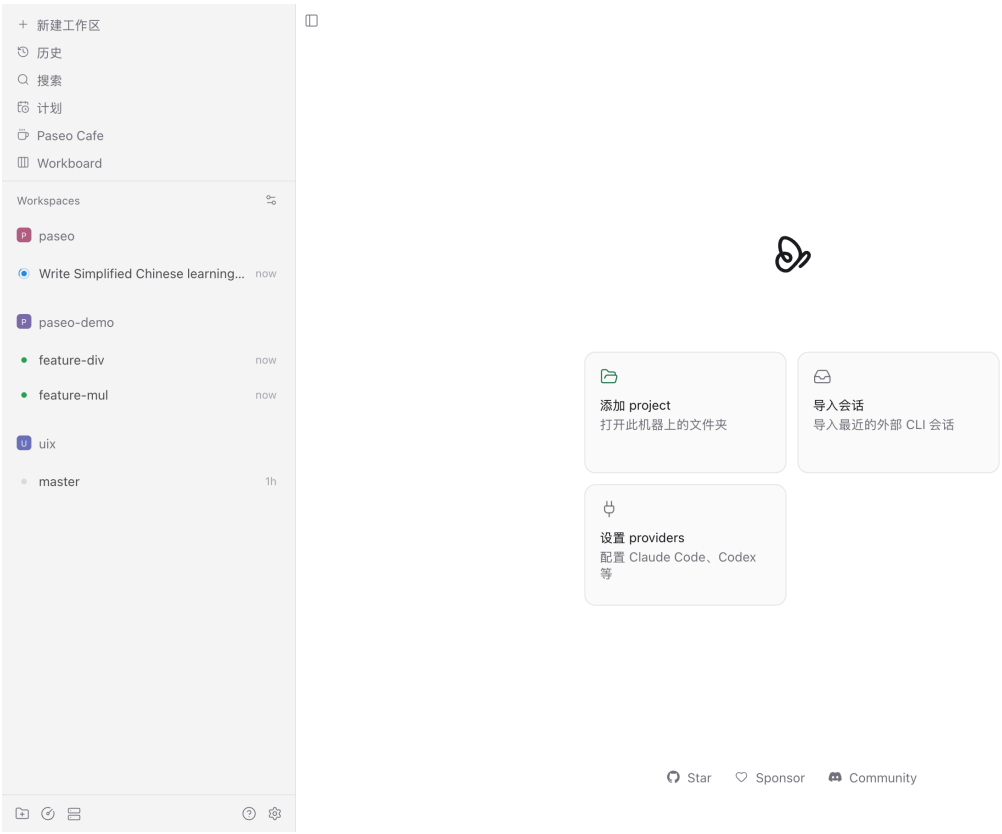
第二，两个 agent 开始运行。`paseo ls` 能看到它们。

```
AGENT ID  NAME                      PROVIDER  STATUS   CWD
9fbd673   Add a divide command...   codex     running  ~/.paseo/worktrees/01qoxyco/feat
bc04f4e   Add a multiply command..  codex     running  ~/.paseo/worktrees/01qoxyco/feat
```

注意 CWD 这一列。两个 agent 在不同的 worktree 目录里干活。

> **重点看**：CWD 列的 worktree 路径。
>
> `~/.paseo/worktrees/01qoxyco/feature-mul` 和 `feature-div`。`01qoxyco` 是源仓库路径的哈希，后面是各自的 slug。这就是隔离的物理位置。

在 Web UI 里看，两个工作区同时出现在侧边栏，各占一个 worktree：



图：侧边栏里 `paseo-demo` 项目下，`feature-mul` 和 `feature-div` 两个工作区并列。这就是并行开发的样子。

3.4 审批：agent 改文件前要问一下

第一次跑，有个环节会卡住你：权限审批。

两个 agent 停在了 `permission` 状态。它们要改文件，但 codex 默认模式下，写文件需要审批。

查一下待审批项：

```
paseo permit ls
```

预期结果：每个 agent 一条 `CodexFileChange` 请求。

批准：

```
paseo permit allow <agent-id>
```

> **鸟窝的经验**：第一次看到 permission 状态别慌。
>
> 这是 agent 在等你点头，不是卡死了。用 `paseo permit ls` 看，`paseo permit allow <id>` 批。机器上装过几种 agent 的，模式不同，有的问得多，有的不问。

3.5 结果：两个 agent 各交各的作业

两个 agent 都完成了。看看它们干了什么。

第一个，feature/mul：

```
def mul(a, b):
    return a * b
```

第二个，feature/div：

```
elif op == "div":
    print(a // b)
```

重点是它们各自的测试汇报。

第一个的日志：

```
[Shell] python3 calc.py mul 6 7
Added a `mul` command and `mul(a, b)` helper to calc.py, with usage updated.
Test output:
python3 calc.py mul 6 7
42
```

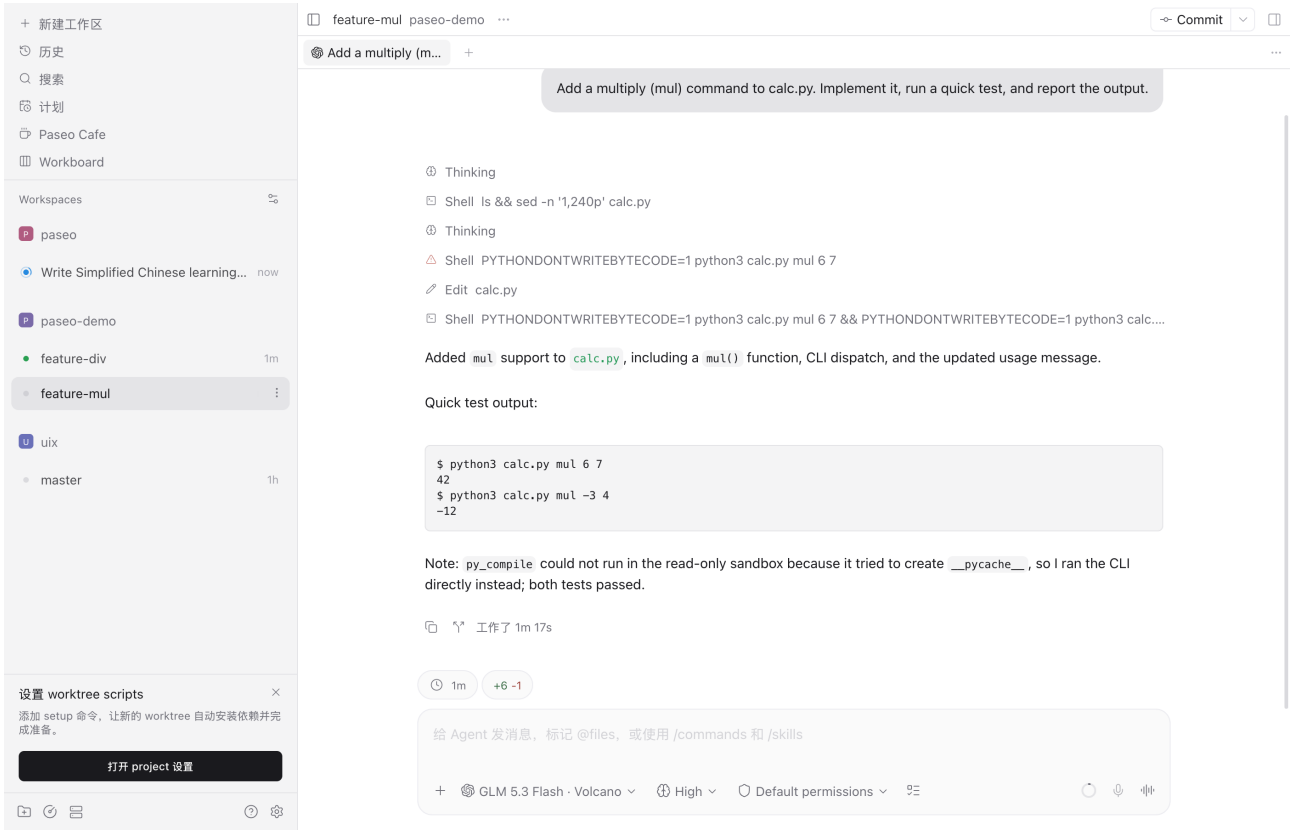
第二个的日志：

```
[Shell] python3 calc.py div 20 4
Added a `div` command to calc.py, implemented as integer division.
Test output:
python3 calc.py div 20 4
5
```

两个 agent 都自己改了文件、自己跑了测试、自己汇报了结果。42 和 5，都对。

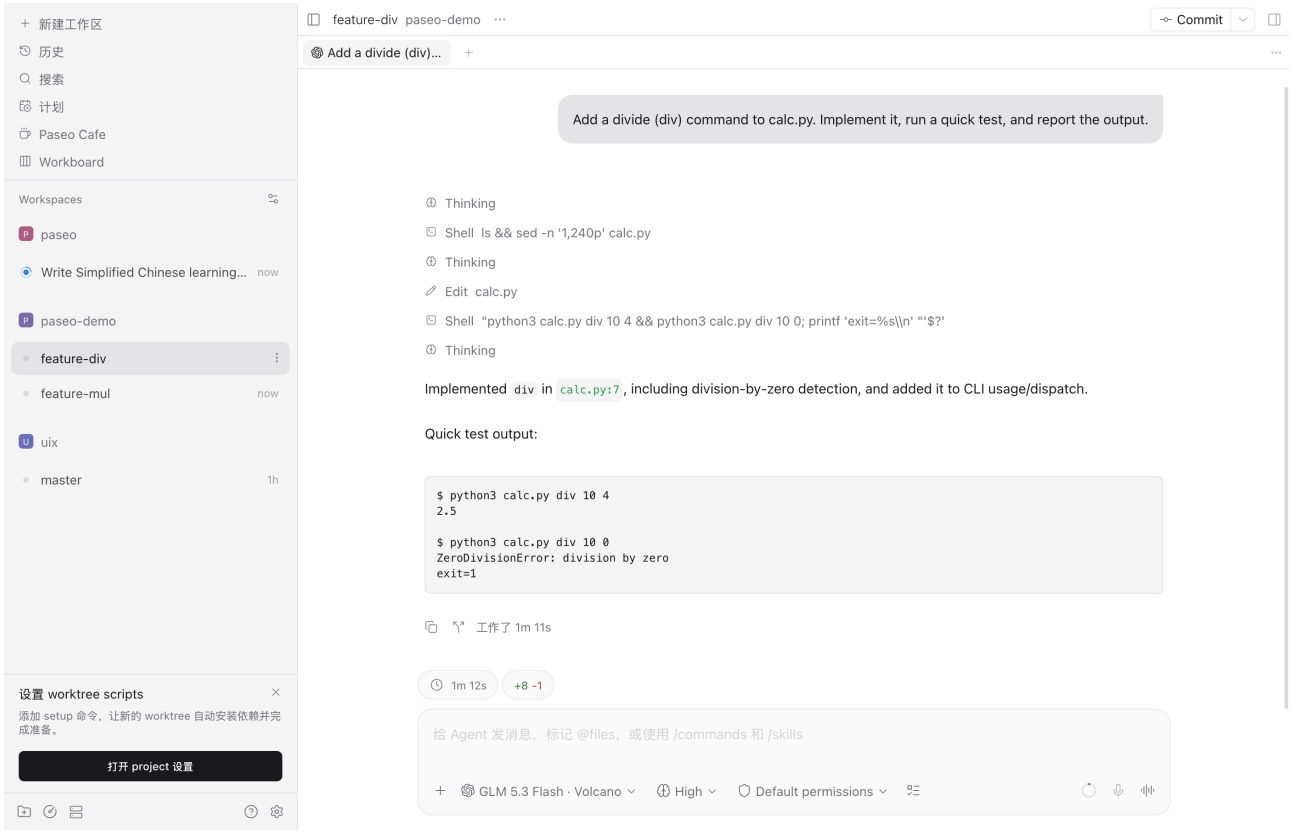
> ⚠ **注意**：agent 改了文件，但不一定提交。
>
> 我的两个 agent 都停在"改完未提交"的状态。这是常见情况。任务没让它们提交，它们就不提交。审查 diff，然后由你（或让 agent）提交。别默认 agent 会做完整个 git 流程。

这是我重新跑了一遍 demo 时，在 Web UI 里截下的两个 agent 的真实对话。第一个 agent（乘法）：



图：*feature-mul* 工作区里，*agent* 依次执行了 *Thinking* → *Shell*（读文件）→ *Shell*（跑测试）→ *Edit*。最后汇报：`mul 6 7` 输出 42。右下角能看到工作时长 1*m 17s* 和 *diff* 统计 +6 -1。

第二个 agent（除法）：



图：*feature-div* 工作区里，*agent* 实现了 `div`，还自己测了除零：`div 10 0` 抛 `ZeroDivisionError`。它连边界情况都想到了。

两幅图对比看，能体会"并行"：两个 agent 各干各的，互不知道对方存在。它们的改动，最后要在 main 上汇合。

3.6 审查、合并与冲突

文件改了，测试过了。现在合并回主分支。

第一步，审查 diff。`git diff` 看两个 worktree 各自改了啥。前面 `cat` 已经看过，改动很干净。

第二步，提交。两个 worktree 各自 commit。

```
[feature/mul 164e623] feat: add mul command
[feature/div f2c7d99] feat: add div command
```

第三步，合并到 main。先合并 mul：

```
Merge made by the 'ort' strategy.
 calc.py | 7 ++++++-
 1 file changed, 6 insertions(+), 1 deletion(-)
```

干净。再合并 div：

```
Auto-merging calc.py
CONFLICT (content): Merge conflict in calc.py
Automatic merge failed; fix conflicts and then commit the result.
```

冲突了。

为什么？两个 agent 都改了同一段代码。`mul` 和 `div` 都要插进同一个 `elif` 分支链，还要改同一行 usage 字符串。worktree 隔离了"并行干活"，但没隔离"合并到同一条基线"。

这是正常的。并行开发的代价，就是合并时可能冲突。解决方式跟平时一样：打开文件，两边都保留，提交。

我保留了双方：

```
elif op == "sub":
elif op == "mul":
elif op == "div":
```

合并提交后，验证：

```
python3 calc.py mul 6 7   # 42
python3 calc.py div 20 4  # 5
python3 calc.py add 1 2   # 3
```

全过。

3.7 清理：归档工作区

收尾。归档两个工作区。

```
paseo workspace archive wks_1ff7562651a0c716
paseo workspace archive wks_f9e6cd4293ffb253
```

预期结果：归档后，受管的 worktree 目录被删除。

```
=== worktrees dir after archive ===
（空）
```

生命周期完整：创建 worktree → 并行干活 → 审查合并 → 归档清理。

3.8 回顾

这一章，我们用真实案例跑通了 Paseo 的核心流程。

四件事：worktree 隔离并行、权限审批、审查合并、归档清理。

> **核心建议**：记住"谁提交"。
>
> 并行 agent 干完活，改的文件在各自 worktree 里，未提交。审查、提交、合并、解决冲突，都是你（或主 agent）的职责。Paseo 管并发，git 管版本，你管决策。

到这里，第一层跑通了：装好、跑通、并行。下一章开始深入核心能力——先把工作区模型讲透。

Part 2: 核心能力

深入 Paseo 的关键能力，每章一个核心概念。

第四章 Workspaces：Paseo 的工作模型

先给结论：Paseo 围绕工作区组织，不围绕聊天组织。

这是它和聊天式 AI 工具最本质的区别。其他工具里，你的单位是"对话"。Paseo 里，你的单位是"工作区"。

官方定义一句话：A workspace is the place where a task happens. 工作区，是任务发生的地方。我把这句话刻在脑子里，比任何参数都重要。

4.1 围绕工作区，不是围绕聊天

想想你用聊天式工具的体验。对话列表越来越长。每条对话里，prompt、代码、测试结果混在一起。任务做到一半，要换个模型，得开新对话，上下文全丢。

Paseo 反过来。工作区是稳定的容器，会话是放在容器里运行的东西。

一个工作区可以同时装：多个 agent 会话、终端、浏览器、diff 面板。

你在工作区里开一个 agent 跑前端。同一时间，同一个工作区里再开一个 agent 审查代码。再开一个终端跑测试。三个东西共享同一个工作目录。

这个模型解决了一个核心问题：任务是持续的过程。不是一次性的对话。

我把工作区当成任务的文件夹。这个习惯，帮我管住了所有并发的活。

> **鸟窝的经验**：把"对话"换成"工作区"，一开始不习惯。
>
> 我用 Paseo 的头几天，还在用聊天的思维找对话记录。后来才明白：工作区才是任务的容器。对话死了，任务没死。

4.2 项目 → 工作区 → 会话

三层结构。

第一层，项目（Projects）。它对应一个代码目录。可以是 git 仓库，可以是 GitHub 项目，也可以是 daemon 机器上的任何目录。

第二层，工作区（Workspaces）。任务发生的地方。一个项目可以有多个工作区。

第三层，会话（Sessions）。你实际跑起来的 agent 对话。一个工作区可以同时容纳多个会话。

关系像这样：

```
项目（my-app）
├── 工作区 A（main，主分支）
│   ├── 会话 1（Claude 跑测试）
│   └── 会话 2（Codex 改 bug）
└── 工作区 B（feature-login，worktree 隔离）
    └── 会话 3（实现登录）
```

> **重点看**：工作区是"地方"，会话是"活动"。
>
> 工作区有物理位置（一个目录），会话没有。会话只是运行在工作区里的 agent 实例。理解这个，后面所有功能都好懂。

4.3 Local vs Worktree：两种隔离

每个工作区，隔离模式二选一。

Local 隔离。 工作区直接用已有目录。适合会话之间要共享磁盘文件的情况。比如主开发目录。

```
paseo workspace create --isolation local --path ~/dev/my-app --title main
```

预期结果：注册一个本地工作区。它指向 `~/dev/my-app`，不改动任何文件。

Worktree 隔离。 工作区创建或打开一个受管的 git worktree。每个任务有独立目录、独立分支。适合并行开发。

```
paseo workspace create --isolation worktree --path ~/dev/my-app --base origin/main
```

预期结果：Paseo 在受管目录里新建一个 worktree，从 `origin/main` 切出独立分支。

维度	Local 隔离	Worktree 隔离	鸟窝的结论
目录	用现有目录	新建受管目录	Worktree 更干净
分支	共享当前分支	独立分支	Worktree 赢
共享文件	直接可见	要看 setup 配置	Local 更省事
并行安全	会互相踩	互不干扰	Worktree 赢
适用	单任务、主开发	多任务并行	按场景选

> 🎯 **核心建议**：拿不准，用 Worktree。
>
> 并行任务用 Worktree 隔离，天然防冲突。单任务、或者需要和主 checkout 共享文件，用 Local。

4.4 生命周期：工作区不是永久存在的

工作区有完整的生命周期。创建 → 使用 → 归档 → 清理。

一个细节值得注意：多个工作区可以指向同一个受管 worktree。Paseo 在最后一个引用它的工作区被归档后，才删除这个 worktree。我实测过归档流程，目录立刻清空。

我在 第三章 实测过这个流程。两个 worktree 工作区归档后，受管目录立刻清空。

```
paseo workspace ls          # 看活跃工作区
paseo workspace archive <id> # 归档工作区及其所有会话
paseo run --workspace <id> "..."  # 在指定工作区跑 agent
```

还有一件事：创建 agent 和创建工作区，是两件事。

裸跑 `paseo run`，Paseo 会新建一个本地工作区。在已有工作区里跑，用 `--workspace` 指定。agent 内部调用时，Paseo 会在当前工作区里创建 subagent。

4.5 paseo.json：工作区的自动化配置

工作区要跑服务、装依赖，靠的是 paseo.json。它放在仓库根目录。Paseo 从你选定的基础分支的已提交版本读取它。

```
{
  "worktree": {
    "setup": "npm ci",
    "teardown": "rm -rf .cache"
  },
  "scripts": {
    "test": { "command": "npm test" },
    "web": { "command": "npm run dev", "type": "service", "port": 3000 }
  }
}
```

三个部分。

setup。 worktree 创建后跑一次。新 worktree 没有依赖，没有 `.env` 这类被 git 忽略的文件。setup 负责补上。

```
{
  "worktree": {
    "setup": "npm ci\ncp \"$PASEO_SOURCE_CHECKOUT_PATH/.env\" .env\nnpm run db:migrate"
    "teardown": "npm run db:drop || true"
  }
}
```

`$PASEO_SOURCE_CHECKOUT_PATH` 指向原 checkout，方便拷文件。teardown 在归档时、删除目录前跑，负责清理。

scripts。 按需运行的具名命令。`type: "service"` 的脚本会被作为长驻进程监管，获得端口，HTTP 流量经 daemon 反向代理。

terminals。 预配置的终端。工作区一开，这些终端就位。

```
{
  "worktree": { "terminals": [ { "name": "logs", "command": "tail -f dev.log" } ] }
}
```

> **鸟窝的经验**：setup 写得好，并行才省心。
>
> 没有 setup，每个新 worktree 都要手动装依赖。有了 setup，agent 一进去就有完整环境。这个文件值得花时间配。

4.6 实战：把现有仓库变成工作区

真实操作一遍。把我在 第三章 造的 demo 仓库注册成本地工作区。

```
paseo workspace create --isolation local --path /tmp/paseo-demo --title demo-local
```

预期结果：

```
WORKSPACE ID          PROJECT     NAME         ISOLATION  CWD
wks_64e8aac144332a4c  paseo-demo  demo-local   local      /tmp/paseo-demo
```

看全部工作区：

```
paseo workspace ls
```

用完归档：

```
paseo workspace archive wks_64e8aac144332a4c
```

预期结果：

```
WORKSPACE ID          STATUS      ARCHIVED AT
wks_64e8aac144332a4c  archived    2026-10-10T23:21:15.953Z
```

一个工作区从生到死，就这三条命令。

> **核心建议**：给每个任务一个工作区。
>
> 任务开始，建工作区。任务结束，归档工作区。归档不是删除，它软删除，历史还在。这个习惯让 Paseo 的目录永远清爽。

工作区是容器，worktree 是隔离。下一章，把 worktree 的三种模式和端口、代理这些硬核细节讲透。

第五章 Worktree 并行开发：杀手锏

先给结论：worktree 是 Paseo 最实用的能力，没有之一。

第三章 已经演示过一次。这一章，把原理、三种模式、服务和端口讲透。

5.1 原理：独立目录 + 独立分支

git worktree 不是新东西。它是 git 的原生功能。一个仓库，多个工作目录，各占一个分支。

Paseo 把它包装成产品能力。你只需要说"给我一个 worktree"，剩下的它管。

worktree 默认放在这里：

```
~/.paseo/worktrees/
└── 1vnnm9k3/                # 源仓库路径的哈希
    ├── tidy-fox/            # worktree slug
    └── bold-owl/
```

两层结构。第一层，源 checkout 路径的哈希。第二层，worktree 自己的 slug。我第一次看到这个目录结构，就明白它为什么这么设计。

想改位置？配置 `worktrees.root`：

```
{ "worktrees": { "root": "/mnt/fast/paseo-worktrees" } }
```

> **重点看**：哈希分组。
>
> 同一个仓库的所有 worktree 分在同一组。不同仓库各占一组。找文件，先看哈希，再看 slug。

5.2 三种模式

`paseo run` 和 `paseo workspace create` 都支持三种 worktree 模式。区别在于"以什么为起点"。

branch-off：从基础分支切新分支。 最常用。我书里的例子，几乎全是这种。

```
paseo run --new-workspace worktree \
  --worktree-mode branch-off \
  --new-branch feature/auth \
  --base origin/main \
  "implement auth"
```

从 `origin/main` 切出 `feature/auth`。agent 在全新分支上干活。

一个细节：base 用 `origin/main`，别用 `main`。Paseo 后台抓取远端 ref，远端跟踪分支永远最新。本地 `main` 可能过期。

checkout-branch：检出已有分支。 任务接着已有分支干。

```
paseo run --new-workspace worktree \
  --worktree-mode checkout-branch \
  --branch feature/existing \
  "continue the work"
```

checkout-pr：以 PR 为工作区。 任务针对某个 PR。

```
paseo run --new-workspace worktree \
  --worktree-mode checkout-pr \
  --pr-number 2186 \
  "review this PR"
```

Paseo 无法从源 checkout 推断 forge 时，加 `--forge <name>`。

模式	起点	适用	鸟窝的结论
branch-off	新分支	新任务	默认选它
checkout-branch	已有分支	接续任务	中途接手用
checkout-pr	PR	审 PR、修 PR	团队协作用

> **鸟窝的经验**：branch-off 是 80% 场景的答案。
>
> 新任务、新功能、新修复，全部 branch-off。checkout-branch 和 checkout-pr 是特定场景的补充。先把 branch-off 用熟。

5.3 服务与端口：让 agent 跑起开发服务器

worktree 里跑开发服务器，是常见需求。Paseo 有一套服务机制。

在 paseo.json 里配置服务：

```
{
  "scripts": {
    "web": { "command": "npm run dev", "type": "service", "port": 3000 }
  }
}
```

`type: "service"` 表示长驻进程。Paseo 监管它，给它分配端口，HTTP 流量经 daemon 反向代理。

省略 `port`，自动分配。进程绑定环境变量 `$PASEO_PORT`。多个 worktree 副本，各占各的端口，不打架。

端口范围可以配置：

```
// ~/.paseo/config.json
{ "worktrees": { "servicePorts": { "range": "3000-4000" } } }
```

服务通过确定性主机名访问：

```
http://web--my-app.localhost:6767
```

格式：`<脚本名>--<项目名>.localhost:<daemon端口>`。有分支时，中间加分支段。`*.localhost` 在现代系统上解析到 127.0.0.1。代理支持 WebSocket。

服务间还能互相访问。同一个工作区里，web 和 api 两个服务，互相关心对方地址：

```
PASEO_PORT=3000
PASEO_URL=http://web--my-app.localhost:6767
PASEO_SERVICE_API_PORT=51732
PASEO_SERVICE_API_URL=http://api--my-app.localhost:6767
```

> ⚠ **注意**：这段服务机制，我从文档读到，没实测。
>
> 我的 demo 项目没有开发服务器，没跑到服务这一步。命令和 URL 格式均来自官方文档。你的项目有 dev server 时，按上面的格式配置。

5.4 实战：一个仓库，多个任务

第三章 已经演示了双任务并行。这里讲怎么放大到多个任务。

三个任务：加乘法、加除法、加取模。三条命令几乎同时发出。

```
# 任务 1
paseo run --new-workspace worktree --worktree-mode branch-off \
  --new-branch feature/mul --worktree-slug feature-mul --base main \
  --provider codex "add mul command to calc.py"

# 任务 2
paseo run --new-workspace worktree --worktree-mode branch-off \
  --new-branch feature/div --worktree-slug feature-div --base main \
  --provider codex "add div command to calc.py"

# 任务 3
paseo run --new-workspace worktree --worktree-mode branch-off \
  --new-branch feature/mod --worktree-slug feature-mod --base main \
  --provider codex "add mod command to calc.py"
```

三个 agent 同时跑。各自独立目录、独立分支、独立测试。

`paseo ls` 能看到全部三个。侧栏切换，看各自进度。

要不要给每个任务配不同模型？ 可以。`--provider claude/opus-4.6` 和 `--provider codex/gpt-5.5`，各任务各挑。这就是"按任务混用 provider"。

5.5 冲突与清理

并行开发的代价，合并时收。

我在 第三章 实测过：两个 agent 改同一段代码，第一个合并干净，第二个冲突。

冲突不可怕。这是 git 的正常行为。worktree 保证了"干活时互不干扰"，不保证"合并时零冲突"。两个 agent 都改同一段代码，合并就冲突。

解法：解决冲突时两边都保留，然后提交。

清理：任务结束，归档工作区。Paseo 自动删除受管的 worktree。

> ◎ **核心建议**：让 agent 各自负责不重叠的文件。
>
> 分配任务时，尽量按文件分区。你改 `auth.py`，我改 `api.py`，合并时零冲突。这是并行开发的黄金法则。

5.6 回顾

worktree 这一章的核心：

第一，独立目录 + 独立分支，并行不踩踏。第二，三种模式，branch-off 为主。第三，服务机制，让 agent 跑真服务器。第四，冲突在合并时收，按文件分区可避免。

> **一句话总结**：worktree 隔离干活，git 合并收尾，按文件分区零冲突。

上面讲的是"怎么隔离并行"。下一章讲"谁在并行"——多 provider 编排，让不同的 agent 协作。

第六章 多 Provider 编排

先给结论：Paseo 不选边站。它让所有编码 agent 在同一个界面里协作。

Claude Code 是 Claude Code，Codex 是 Codex。Paseo 是它们共同的调度层。

6.1 Provider 抽象：你装 CLI，Paseo 运行它

官方原则一句话：你安装底层 CLI。Paseo 运行它。

Provider 是 Paseo 与某个外部 agent CLI 之间的契约。契约定义了四件事。启动、输出、回传、模式，每件都有约定。我靠一条模型 ID，就能选中整套契约。

关键点：底层二进制活在你的机器上，以普通子进程运行。你的订阅、凭证、配置、skills、MCP server，全部原样保留。

Paseo 不碰你的账号。它只是把 CLI 进程管起来。我跑 claude 和 codex，各自的登录都原样在用。

> **鸟窝的经验**：这个设计很关键。
>
> 我用 Paseo 跑 claude 和 codex，各自的登录状态都在。同一个界面，两套模型，零配置切换。第一次跑：claude 4 秒完成。第二次换 codex，6 秒。谁便宜用谁，谁快用谁。

6.2 原生 6 个 + ACP 36 个

Paseo 的 provider 分两个层级。

原生支持。 内置适配器，装好 CLI 就自动发现。

Provider	说明
Claude Code	Anthropic 的编码 agent，支持 MCP、流式、深度推理
Codex	OpenAI 的 workspace agent，带沙箱控制
OpenCode	开源编码助手，支持多 provider 模型
Pi	极简终端编码 agent
Antigravity	通过内置插件驱动 `agy` CLI
Muse Code	Meta 的终端编码 agent

ACP 目录。 任何讲 Agent Client Protocol 的 agent，都能通过通用适配器工作。Paseo 提供精选目录，一键安装。

目录里有 36 个。挑几个眼熟的：Cursor、Gemini CLI、GitHub Copilot、Hermes Agent、Kimi Code CLI、Qwen Code、TRAE CLI、GLM Agent、goose、Grok。我机器上装了 4 个 provider，全被自动发现，一个没漏。

> **重点看**：ACP 这个标准。
>
> Agent Client Protocol 是 agent 之间的通用语言。Paseo 赌的是这个标准。目录外的 agent，也支持手动添加。你的机器装了什么 agent，Paseo 就能管什么 agent。

一个特殊案例：Antigravity。它没有权限询问机制。Paseo 启动它时，跳过权限询问。每次工具调用直接执行。会话开头显示警告。文档原话：未经 Google 认可。用之前想清楚。

6.3 模型 ID：provider/model

在 Paseo 里，模型用一条 ID 表示。格式：provider 斜杠 model。

```
claude/opus-4.6
codex/gpt-5.5
opencode/openrouter/glm-5.1
```

`--provider` 参数接受两种写法。只写 provider，用它的默认模型。写全，指定模型。我自己习惯写全，心里有数。

```
paseo run --provider codex "fix the bug"
paseo run --provider codex/gpt-5.5 "fix the bug"
```

6.4 Orchestration：让 agent 委派 agent

Orchestration 是 Paseo 的编排能力。核心：agent 能启动其他 agent，跨 provider，跨机器。

agent 能做到的事：

第一，按任何已配置的 provider/模型启动新 agent。第二，把研究、实现、审查拆给多个 agent，可以来自不同 provider。第三，按 ID 给其他 agent 发 prompt。第四，查进度、停任务、收结果。

两条入门路径。

路径一：MCP 工具注入。 默认关闭。开启：Settings → your host → Agents → Enable Paseo tools。开启后，agent 就多了一组工具：`create_agent`、`send_agent_prompt`、`list_agents`，等等。

路径二：CLI。 agent 有 shell 权限就能用。

```
paseo run --provider codex --no-wait "Review this branch without changing files"
```

`--no-wait` 启动 worker 立即返回，不阻塞主 agent。`paseo ls -a` 查看所有 agent，包括后台的。我写脚本时，靠这两条控制节奏。

一个典型的委派 prompt：

> "Use Paseo to launch a second agent to review this branch. Have it report potential bugs without changing files, then summarize its findings."

6.5 Agent profiles：把常用设置存起来

每次启动 agent，都要选 provider、模型、模式、thinking 级别。烦不烦？

Agent profiles 解决这个。把常用组合存成命名 profile，一次选择。

创建：Settings → your host → Agents → Agent profiles → New profile。

三个关键字段。

字段	作用
provider / model / mode / thinking	启动设置
When to use	备注：这个 profile 适合什么任务
保存位置	当前 host

备注字段是灵魂。编排 agent 启动 worker 前，会读所有 profile 的备注，按任务匹配。

Profile	备注示例
UI work	Use for components, layout, styling, and visual polish.
Planning	Use for architecture, investigating root causes, and comparing approaches.
Review	Use for independent review of diffs, correctness, missing tests.

我配了三个 profile，各管一类活。

> **核心建议**：把"什么任务用什么 agent"写进备注。
>
> 备注写得清楚，编排 agent 就能自动选对 profile。备注写不清楚，它只能瞎猜。我把备注当"用人说明"来写。

6.6 实战：Claude 规划 + Codex 实现 + 独立审查

经典三阶段工作流。

阶段一：Claude 规划。 架构、方案、拆分，Claude 擅长。产出实现清单。我把这个阶段叫"定盘子"。

阶段二：Codex 实现。 拿着清单，Codex 动手。便宜，跑得快。

阶段三：独立审查。 换一个 agent（或另一个 profile），在同一个工作区里审查 diff，不改文件。我组这套流程，花了不到一天。

用 agent 间的交接，靠 agent ID。右键目标 agent 标签，复制 agent ID，粘贴到源 agent 对话里，指示它发消息。底层是 `send_agent_prompt` 工具。

```
paseo send <agent-id> "review the diff and list risks"
```

Paseo subagent 和原生 subagent 的区别，一张表说清：

维度	Paseo subagent	原生 provider subagent	鸟窝的结论
Provider	任意已配置 provider	父 provider 自己选	Paseo 更灵活
工作区	当前或显式选择	provider 管理	Paseo 可控
对话	可对话的完整会话	只读时间线	Paseo 可交互
控制	追问、改设置、归档	生命周期归 provider	Paseo 可管理

> **鸟窝的经验**：三阶段工作流，价值在"独立审查"。
>
> 写代码的 agent 容易爱上自己的代码。换一个 agent 审查，能挑出真问题。我实测两个 provider 从同一界面跑通后，就组了这套流程：贵的规划，便宜的干活，第三方审查。

6.7 回顾

这一章的核心：

第一，你装 CLI，Paseo 运行它。第二，原生 6 个加 ACP 36 个，几乎覆盖所有 agent。第三，模型 ID 是 provider/model。第四，编排让 agent 跨 provider 委派。第五，profile 把常用设置存起来。

> **一句话总结**：Paseo 不选边站，它让所有 agent 在同一个界面协作。

到这里，桌面上的事讲完了。下一章，走出桌面——手机、Web、远程机器，跨设备连接。



Part 3: 进阶实战

多设备、多工具、多场景的复杂用法。

第七章 跨设备：手机到云端

先给结论：Paseo 的 agent 跑在你机器上，但你可以从任何设备指挥它们。

桌面、手机、浏览器、终端，全是同一个 daemon 的客户端。我实测：daemon 常驻，relay 开着，任何客户端连上就能用。

7.1 连接模型：daemon / relay / 客户端

先理解三个角色。

我在自己机器上验证过这个模型。daemon 常驻，relay 开着，任何客户端连上就能用。

daemon。 常驻在你机器上的服务。它拥有 agent 和工作区。默认监听 `127.0.0.1:6767`。

客户端。 桌面 app、iOS/Android app、浏览器 Web UI、CLI。全都连接 daemon。

relay。 中继服务器。它解决一个问题：daemon 在内网，手机在外面，怎么连？

三种连接方式。

方式一：relay。 daemon 主动出站连接中继服务器，不需要开放端口。流量端到端加密。适合"机器在家，人在外面"。我的 daemon 就开着 relay。

方式二：SSH。 你本地的 OpenSSH 客户端直连远程 daemon。前提是能 SSH 到那台机器。

```
paseo --host ssh://user@host ls -a
```

方式三：Tailscale 等 VPN。 两台设备在同一个虚拟内网里，直连。

方式	需要开放端口	加密	适合	鸟窝的结论
relay	否	端到端	人在外面	首选
SSH	否	SSH 加密	能 SSH 的机器	服务器首选
Tailscale	否	WireGuard	家里多设备	局域网好用

7.2 手机配对与加密

手机连接的关键动作：配对。

配对用二维码。daemon 打印一个二维码，手机扫一下，就建立了信任关系。

```
paseo daemon pair
```

预期结果：终端打印二维码和配对链接。手机 app 扫码或粘贴链接。

配对背后的加密细节，值得知道。它解释了一个问题：数据经过第三方服务器，为什么安全。

第一步，daemon 生成持久 ECDH 密钥对，存在 `$PASEO_HOME/daemon-keypair.json`。第二步，扫码把 daemon 公钥送到手机。第三步，手机用自己的公钥发握手消息。握手完成前，daemon 不接受任何命令。第四步，双方用 Curve25519 组合出加密通道，载荷用 XSalsa20-Poly1305 保护。

官方 relay 服务器是开源的 Elixir 服务（getpaseo/paseo-relay）。设计目标：relay 不可信。它只能看到 IP 地址、时序、消息大小、会话 ID，看不到内容。

> ⚠ **注意**：配对二维码就是信任锚。
>
> 官方文档原话：把它当密码，别公开分享。谁拿到二维码，谁就能配对你的 daemon。我实测环境里 relay 默认关闭，只有配对设备时才启用。

7.3 Web UI 与远程 daemon

不开 app，纯浏览器也能用。daemon 自己就带一个 Web UI。

默认关闭。开启：

```
paseo daemon config set features.webUi.enabled true
paseo daemon start
```

然后打开 `http://localhost:6767/`。

页面和 daemon 同源。这本书写作时，我实测过：开启后打开 `http://localhost:6767/`，看到的就是 第二章 那张主界面截图。截完图，我把它关回默认状态。路径和细节，来自这次实测和文档。

几个细节。

第一，Web UI 和 daemon 同源伺服。API 在 `/api/*`，WebSocket 在 `/ws`。

第二，静态 UI 文件不设密码（登录页要渲染），但 API 和 WebSocket 要认证。

第三，默认只监听 `127.0.0.1`，也就是只能本机访问。要暴露到局域网或公网，改监听地址：

```
paseo daemon config set daemon.listen 0.0.0.0:6767
paseo daemon start
```

> ⚠ **注意**：改监听地址前，先设密码。
>
> 文档警告：任何人能连到监听地址，就能用你的 agent。先 `paseo daemon set-password`，再改监听。密码保护访问，但不加密流量。要加密，走 relay 或 VPN。

远程 daemon。 从一台机器指挥另一台机器的 daemon。`--host` 参数指过去就行。

```
paseo --host workstation.local:6767 run "run the full test suite"
```

在远程机器上，仓库和 provider 必须在那台机器上可用。

> **鸟窝的经验**：远程 daemon 是"服务器当工头"的玩法。
>
> 家里一台 Mac Mini 跑 daemon，白天在公司，`paseo --host mini.local:6767 run "..."` 让它干活。下班回家，任务跑完了。这就是"你的 agent 永远在岗"。

7.4 Docker 自托管

不想在个人电脑上跑 daemon？用 Docker，跑在服务器上。

```
docker run -d --name paseo \
  -p 6767:6767 \
  -e PASEO_PASSWORD=change-me \
  -v "$PWD/paseo-home:/home/paseo" \
  -v "$PWD:/workspace" \
  ghcr.io/getpaseo/paseo:latest
```

装完打开 `http://localhost:6767`。

三个关键点。

第一，`PASEO_PASSWORD` 必须设。镜像绑定 `0.0.0.0:6767`，端口是暴露的。不设密码，等于把 agent 大门敞开。

第二，镜像不内置 agent CLI。要加：

```
FROM ghcr.io/getpaseo/paseo:latest
USER root
RUN npm install -g @openai/codex @anthropic-ai/claude-code opencode-ai
```

第三，两个卷。`/home/paseo` 存 Paseo 状态和 agent 凭据，`/workspace` 存代码。容器里 daemon 以非 root 的 `paseo` 用户跑（uid/gid 1000:1000）。

完整点的部署用 Compose：

```yaml
services:
  paseo:
    image: ghcr.io/getpaseo/paseo:latest
    container_name: paseo
    restart: unless-stopped
    ports:
      - "6767:6767"
    environment:
      PASEO_PASSWORD: "change-me"
    volumes:
      - ./paseo-home:/home/paseo
      - ./workspace:/workspace
```

```
docker compose up -d
```

> ⚠ **注意**：这段 Docker 内容我没实测。
>
> 本机没装 Docker。命令和配置全部来自官方文档（/docs/docker）。我在本机用的是 CLI 直连 daemon，Docker 路线是给服务器场景的。

7.5 回顾

跨设备这一章，核心是三个连接方式：relay、SSH、VPN。加上 Docker 自托管。

> **一句话总结**：agent 在 daemon 上跑，daemon 在哪，你就能从哪连。

> **核心建议**：先本地，再远程。
>
> 第一次用，别急着上 relay 和 Docker。先在本地跑通 daemon 和 agent，再加手机配对。远程的坑，等本地熟了再踩。

连接搞定了。下一章，让它自己跑——定时任务、MCP 工具、浏览器自动化。

第八章 自动化：Schedules、MCP 与 Browser

先给结论：Paseo 的自动化，让 agent 从"你叫它才动"变成"到点自己动"。

这一章讲三件自动化的武器：定时任务、工具注入、浏览器。

8.1 Schedules：定时任务

Schedule 按 cron 节奏，启动一个全新的 agent，跑你设定的 prompt。

和 heartbeat 的区别先说清：schedule 每次跑都是新 agent；heartbeat 是同一个 agent 反复醒来续命。后面 08.2 细说。

典型场景。 我最想用的，是构建保姆。凌晨它盯着构建，天亮就有结果。

第一，构建保姆。每 5 分钟查一次构建，挂了就修。

```
paseo schedule create --every 5m --name build-watch \
  --provider claude/opus-4.7 --cwd ~/dev/my-app --max-runs 24 \
  "Check the release build. If it failed, inspect logs, fix the cause, and rerun."
```

第二，每日 triage。每天早上扫 GitHub issues 和 PR，汇总要处理的。

```
paseo schedule create --cron "0 9 * * 1-5" --timezone America/New_York \
  --name morning-triage --provider codex/gpt-5.5 --cwd ~/dev/my-app \
  "Review overnight CI failures and summarize anything urgent."
```

第三，隔夜重构。半夜让 agent 慢慢推进重构，天亮验收。

```
paseo schedule create --every 30m --name overnight-refactor \
  --provider codex/gpt-5.5 --cwd ~/dev/my-app --max-runs 16 --expires-in 10h \
  "Continue the refactor. Run the focused checks. Leave a short status note."
```

关键参数。

参数	作用
`--every 5m`	频率预设，编译成 cron
`--cron "0 9 * * 1-5"`	原始 5 字段 cron
`--timezone`	时区，默认 UTC
`--max-runs`	最多跑几次，防止跑不完
`--expires-in 10h`	过期时间
`--run-now`	创建时立刻跑一次

我实测了一个最小 schedule：

```
paseo schedule create --every 1h --name demo-watch --provider codex --cwd /tmp/paseo-de
```

预期结果：

```
ID          NAME         CADENCE           TARGET              STATUS   NEXT RUN
7e394363    demo-watch   cron:0 * * * *    new-agent:codex     active   2026-10-11T00:
```

注意 CADENCE 列：`--every 1h` 被编译成了 `cron:0 * * * *`。Paseo 不存"每 1 小时滚动"这种间隔，统一转成 cron。我实测的 schedule，一分钟就建好，一秒删掉。

管理命令：

```
paseo schedule ls
paseo schedule pause <id>
paseo schedule resume <id>
paseo schedule run-once <id>
paseo schedule delete <id>
```

四种创建方式。 应用内 Schedules 视图（主方式）、对话里直接说让 agent 建、CLI（headless 机器用）、MCP 编程式建。对话里这么说就行：

> "Every weekday at 9am, triage new GitHub issues and PRs and summarize what needs attention."

agent 会经 MCP 帮你建好 schedule。

> 🎯 **核心建议**：`--max-runs` 和 `--expires-in` 一定要设。
>
> 定时任务最怕失控：跑 1000 次，烧光额度。设上限，agent 自己会停。我的 build-watch 例子设 24 次，够盯半天构建。

8.2 Heartbeats：让 agent 续命

Heartbeat 是另一种定时。它不启动新 agent，它周期性地提醒同一个 agent：继续干活。

场景：一个大型迁移，一个 agent 干不完。让它每 10 分钟醒来一次，继续推进，每次跑完聚焦检查。

Heartbeat 在 agent 会话内创建。需要 `PASEO_AGENT_ID`，这个环境变量在 agent 会话里自动设置。我在 shell 里试不了，但原理我懂。

```
paseo heartbeat create --cron "*/20 * * * *" --name heartbeat \
  "Check the current task state and continue with the next useful step."
```

管理：

```
paseo heartbeat update <id> --cron "*/10 * * * *"
paseo heartbeat delete <id>
```

一个限制值得注意：更新 heartbeat 只能改节奏和时区。目标和 prompt 固定。要改内容，删除重建。这是有意的设计——防止 heartbeat 悄悄变成另一个任务。

> ⚠ **注意**：这段我没实测。
>
> heartbeat 需要在 agent 会话里跑，我在 shell 里没法复现。命令与行为来自官方文档。schedule 和 heartbeat 共用同一个 cron 引擎，只是产品形态不同。

8.3 MCP：Paseo 工具目录

MCP 是 agent 工具的标准协议。Paseo 把它的编排能力包装成 MCP 工具，注入 agent。

两个开关。

```
{
  "daemon": {
    "mcp": {
      "enabled": true,
      "injectIntoAgents": false
    }
  }
}
```

`enabled` 默认 true：运行 MCP server。`injectIntoAgents` 默认 false：工具不注入 agent。

要编排，打开第二个：

Settings → your host → Agents → Enable Paseo tools

改了配置，新开 agent 或重载已有 agent 才生效。运行中的会话保留启动时收到的工具目录。

工具目录长什么样？ 按类别分。我把目录过了一遍，最常用的是 create_agent。

类别	工具
Agents	create_agent、send_agent_prompt、list_agents、get_agent_status、cancel_agent、archive_agent、kill_agent、update_agent
Workspaces	create_workspace、list_workspaces、rename_workspace、archive_workspace
Terminals	create_terminal、list_terminals、capture_terminal、send_terminal_keys、kill_terminal
Schedules	create_schedule、list_schedules、pause_schedule、resume_schedule、delete_schedule
Profiles	list_profiles
Providers	list_providers、list_models、inspect_provider
Permissions	list_pending_permissions、respond_to_permission
Browser	browser_snapshot、browser_click、browser_fill 等
Voice	speak

一个心智模型，来自官方文档。Workspaces 决定工作发生在哪里。agent parentage 决定谁拥有工作。

`create_agent` 不带 workspaceId，subagent 就在自己的工作区。带了，就放指定工作区，但父子关系不断。

按 provider 限制工具。 全局开了注入，还可以给特定 provider 收窄：

```
{
  "agents": {
    "providers": {
      "codex-worker": {
        "extends": "codex",
        "paseoTools": { "disabledTools": ["create_agent", "send_agent_prompt", "kill_ag
      }
    }
  }
}
```

> ⚠ **注意**：这个设置不是安全边界。
>
> 文档原文：它限制"呈现给 agent 的目录"，不是"能经 shell 访问主机的 agent"的安全边界。agent 有 shell，绕开工具照样能乱来。信任要建立在 agent 本身。

8.4 Browser：agent 自己开浏览器验证

最实用的自动化，放最后。agent 能驱动真实浏览器标签。

不是"agent 说你改动应该有效"。是 agent 自己打开页面，亲眼检查。

典型用法。

验证自己改动：开 dev server，快照页面，确认新文本、新布局在。

复现 bug：按 bug 报告点同样的操作序列，读 console 和网络日志。

走完整流程：表单、多步向导、hover 菜单、拖放、文件上传。

已登录会话：标签保留会话状态。你登一次，agent 就能在登录后工作。

开启。 默认关闭。两个开关都要开。我的配置里，浏览器工具已经开了：

Settings → your host → Agents → Browser tools
Settings → your host → Agents → Enable Paseo tools

或 config：

```
{ "daemon": { "browserTools": { "enabled": true } } }
```

主工具：browser_snapshot。 它返回的不是 HTML，是可访问性树。标题、文本、表单状态、层级。

```
- document "Settings"
  - heading "Workspace" [level=1]
  - textbox "Display name" [ref=@e2]
  - button "Save changes" [ref=@e3]
```

交互元素带 ref。agent 把 `@e3` 传给 `browser_click`、`browser_fill`。

ref 来自该标签最新快照。页面变了，ref 就失效，报错而不是作用到错误元素。这个设计防误点。

架构。 agent 经 MCP 连 daemon，daemon 做 broker，把工具调用路由到承载浏览器标签的桌面 app。

```
agent ──MCP──▶ daemon (broker) ──▶ 桌面 app (browser host) ──▶ webview
```

细节：daemon 自己不跑浏览器。桌面 app 没连上，浏览器工具就返回错误。标签按工作区隔离。点击、按键、hover，作为真实浏览器事件派发。页面分辨不出 agent 和用户的点击。

> **鸟窝的经验**：前端闭环，是这套工具最值钱的地方。
>
> 我见过太多"代码写了但没验证"的 agent 输出。浏览器工具让 agent 自己验证自己。配置一次，后面每个改动都能自动过一遍真浏览器。省下的时间，是每次手工刷新页面的时间。

8.5 回顾

自动化三件套：

第一，schedules 定时跑新 agent。第二，heartbeat 让 agent 续命。第三，MCP 注入编排工具，browser 让 agent 自己看页面。

> **一句话总结**：到点自己跑，醒了接着干，干活自己验。

这一章是"用 Paseo"。下一章是"改 Paseo"——skills、插件、SDK。

第九章 扩展：Skills、插件与 SDK

先给结论：Paseo 的三层扩展，从"教 agent 干活"到"改 Paseo 本身"到"把 Paseo 嵌进你的程序"。

三层递进：skills 教 agent，插件改产品，SDK 写程序。我从上往下，一层层试过。

9.1 Skills：让 agent 编排 agent

Skills 给 agent 可复用的委派、交接和审查指令。装一次，任何 agent 对话里都能用。

安装：

```
npx skills add getpaseo/paseo
```

或者应用内：Settings → your host → Agents → Orchestration skills。

我装好以后，日常用 /paseo-handoff 交活。

四个核心 skill。

/paseo。 Paseo Reference，基础 skill。其他 skill 依赖它。它教 agent 委派前先查 agent profiles 和备注。我把它当成入门的必修课。

/paseo-handoff。 任务交接。带上下文把任务交给另一个 agent。交接 briefing 包含：相关文件、进度、决定、约束、验收标准。

> "/paseo-handoff hand off the auth fix to an implementation agent in its own worktree"

/paseo-committee。 委员会。让两个 agent 独立分析同一个难题，互不商量。委员会成员返回分析，不改文件。主 agent 综合方案。

> "/paseo-committee why are the websocket connections dropping under load?"

/paseo-advisor。 顾问。就设计、diff、问题征求另一个 agent 的第二意见。顾问不改文件。

> "/paseo-advisor did I miss anything in this migration plan?"

Skill	干的事	改文件吗	鸟窝的结论
/paseo	教 agent 用 Paseo	-	必装
/paseo-handoff	交接任务	接收方改	团队交接用
/paseo-committee	双 agent 独立分析	不改	难题破局用
/paseo-advisor	第二意见	不改	日常审查用

> **鸟窝的经验**：committee 是意外之喜。
>
> 让两个 agent 独立分析同一个 bug 原因，结果互相印证或互补。比单个 agent 一条道走到黑，稳得多。它的前提是"不改文件"，所以两个 agent 只管想，不怕互相覆盖。

9.2 插件：改 Paseo 本身

插件是安装到一个 daemon 的 TypeScript 项目。它能加屏幕、侧边栏、工作区面板、斜杠命令、主题，甚至接入新的 agent provider。

插件代码跑在 daemon 上。每个连到 daemon 的客户端，都能看到插件的界面。手机上也能。

安装。 v0.11.0 上线了插件注册表。安装用一条命令：

```
paseo plugin add owner/slug
```

从 GitHub 直接装：

```
paseo plugin add github:owner/repo
```

本地目录：

```
paseo plugin add ./path
```

我实测了本机的插件列表。Source 列清楚显示来源：

```
PLUGIN             STATUS    SOURCE                                          REVISION
mermaid            disabled  plugins.paseo.sh/dutchakdev/mermaid             0.2.0
playwright-pulse   disabled  plugins.paseo.sh/stevecastaneda/playwright-pulse  0.1.1
progress           disabled  plugins.paseo.sh/stevecastaneda/progress        0.3.4
```

来源格式：`plugins.paseo.sh/<作者>/<插件名>`。版本号直接列在 REVISION 列。我看了本机插件列表，全是注册表装的。

写插件。 我还没动手写过，但脚手架一条命令：

```
paseo plugin init /absolute/path/to/workspace-plugin
cd /absolute/path/to/workspace-plugin
npm install
```

生成的标准结构：

```
paseo-plugin.json
index.client.tsx      # 客户端入口，跑在 app 里
index.server.ts       # daemon 端入口
client/               # 只编译进 app bundle
server/               # 只编译进 daemon bundle
shared/               # 两端共享
```

每个入口 default-export 一个函数，注册贡献，返回清理函数。

插件能钩哪些事件？ daemon 端生命周期钩子：

`agent.created`、`agent.closed`、`agent.turn_started`、`agent.turn_ended`、`agent.permission_requested`、`agent.permission_resolved`、`workspace.created`、`workspace.archived`

安全。 一句话说透：

> 插件是受信的、不沙箱的代码。服务端代码以 daemon 用户的权限运行，客户端代码跑在 Paseo app 里。安装插件，等于信任那个代码库和它的所有依赖。

> ⚠ **注意**：我只读过插件文档，没写过插件。
>
> 本机装了几个现成插件（mermaid、progress 等），跑通了安装和列表。写插件这部分，命令和 API 均来自官方文档。想写，从 `paseo plugin init` 开始。

9.3 TypeScript SDK：把 Paseo 嵌进你的程序

最硬核的一层。TypeScript SDK 让你用程序控制 Paseo。

安装：

```
npm install @getpaseo/client
```

需要 Node.js 22 或更新。

连接 daemon。 WebSocket，端口 6767。

```
import { createPaseoClient } from "@getpaseo/client";

const client = createPaseoClient({ url: "ws://127.0.0.1:6767/ws" });
await client.connect();
```

创建 agent。 核心 API。

```
const agent = await client.agents.create({
  config: { provider: "codex/gpt-5.5" },
  cwd: "/Users/me/dev/storefront",
  prompt: "Review the current diff and name the riskiest change.",
});
```

`config.provider` 永远是 `provider/model` 格式。`cwd` 是工作目录，Paseo 在背后创建 workspace。

这里有个我踩过的坑：SDK 严格校验格式。我第一版写 `provider: "codex"`，直接报错：

```
Error: Expected config.provider in "provider/model" format
```

必须写全，比如 `claude/claude-opus-5-5`。

等结果。 四种状态。

```
const result = await agent.waitForFinish();
if (result.status === "idle") {
  console.log(result.lastMessage);
}
```

`waitForFinish()` 默认最多等 10 分钟，可传毫秒改截止时间。

状态	含义
idle	回合完成，可接下一个 prompt
permission	在等人批权限
error	provider 出错结束
timeout	超时，agent 还在跑

程序化的典型场景。 定时评审：

```
const client = createPaseoClient({ url: "ws://127.0.0.1:6767/ws" });
await client.connect();

const agent = await client.agents.create({
  config: { provider: "codex/gpt-5.5" },
  cwd: "/Users/me/dev/storefront",
  prompt: "Review the current diff and name the riskiest change.",
});

const result = await agent.waitForFinish();
console.log(result.lastMessage);

await client.close();
```

这段代码，我原样跑过。真实输出：

```
connected to daemon
status: idle
lastMessage: sdk-ok
closed
```

注意：`client.close()` 只是断开连接。断开后，agent 继续在 daemon 上跑，保持可见。之后可以按 ID 恢复：

```
const agent = client.agents.ref("agent_01H8X...");
await agent.refresh();
await agent.run("Now write the fix.");
```

结构化输出。 让 agent 输出 JSON schema 规定的数据：

```
const agent = await client.agents.create({
  config: { provider: "claude/claude-sonnet-5" },
  cwd: ".",
  outputSchema: {
    type: "object",
    properties: {
      summary: { type: "string" },
      risk: { type: "string", enum: ["low", "medium", "high"] },
    },
    required: ["summary", "risk"],
    additionalProperties: false,
  },
  prompt: "Review the diff and report risk.",
});
```

> 🎯 **核心建议**：把 Paseo 当成一个 API 服务用。
>
> 你的 CI、你的脚本、你的内部工具，都可以调 `@getpaseo/client` 派 agent。SDK 不是给交互用的，是给自动化用的。定时跑、事件驱动、批量审查，都是它的主场。

9.4 回顾

三层扩展，一条链路：

skills 让 agent 互相配合，插件让 Paseo 长出你要的界面，SDK 让 Paseo 变成你系统的一部分。

> 💬 **一句话总结**：装 skill 用起来，装插件长出来，写 SDK 嵌进去。

工具讲完了。最后一章，讲人——为什么一个人能靠一群 agent 干活，以及你的下一步。

第十章 心态转变：一个人，一群 agent

先给结论：Paseo 不只是工具，它是一套工作方式的开关。打开它，你从"写代码的人"变成"管 agent 的人"。

这一章不讲命令，讲人。

10.1 时间线：一年，2 万 star

先看 Paseo 自己的故事。这是一个人的项目。

2025 年 10 月 13 日，仓库创建。项目开始于秘密开发。

2026 年 2 月 11 日，第一个公开版本 v0.1.2 发布。项目正式亮相。

2026 年 2 月到 10 月，201 个版本。2026 年 4 月单月 46 个版本。平均每天超过一个。

2026 年 10 月 11 日，我查数据：20,414 个 star，2,409 个 fork。Apache-2.0。

维护者：一个人。Twitter 是 @moboudra，GitHub 是 boudra。靠 GitHub Sponsors 资助。README 里有一行，很诚实："我是独立维护者，不一定每天都能及时处理 GitHub Issues。紧急情况去 Discord。"

> **重点看**：201 个版本这个数字。
>
> 一个人，8 个月，201 个版本。这背后就是 Paseo 自己倡导的工作方式：一个主人，一群 agent 并行干活。它不是在教你怎么用工具，它自己就是这么造出来的。

这个数字对我有什么意义？

工具能不能长期用，看它迭代快不快。一个一年 2 万 star、月更 20 版的工具，说明作者活着，社区活着，协议活着。这是选择工具时，比任何功能都重要的信号。

10.2 我的工作流

写这本书之前，我把 Paseo 完整跑了一遍。这一节是我的真实记录。

第一步：装。 npm 全局装 CLI，v0.11.2。daemon 已经在我机器上跑着。端口 6767。

第二步：验。 跑第一个 agent 任务。claude 的 opus 模型，4 秒完成，回了一行字。再跑一个 codex，6 秒。

第三步：并行。 造了一个计算器 demo。两个 agent 并行开工，一个加乘法，一个加除法。各自的 worktree，各自的测试，各自的汇报。42，5，都对。

第四步：收尾。 合并。第一个干净，第二个冲突。解决，验证，归档。worktree 自动清理。

第五步：自动化。 建了一个定时任务，一小时一次。看到 `cron:0 * * * *` 编译出来，删掉。

这套流程跑下来，我的感受是三个。

第一，**daemon 常驻，是心态的分水岭。** 以前我打开 IDE 才叫"开始干活"。现在 daemon 一直在，agent 一直在，随时可以给它派活。从"我在电脑前"到"我的机器在岗"。

第二，**多 provider，是编排的核心。** 一个界面，claude 和 codex 随便切，不换配置。任务给哪个 agent 干，我说了算。

第三，**worktree，是并行心理学的解药。** 以前怕并行，怕冲突。现在每个任务独立目录，改了不慌，合并时再算账。心理负担小了一半。

> **鸟窝的经验**：从"写代码"到"管 agent"，转变是渐进的。
>
> 第一步，让 agent 干活，你审查。第二步，让 agent 互相配合，你定流程。第三步，让 agent 定时干活，你做决策。我写这本书的过程中，就是这三步。每一步都让"我"的定位往上游移一格。

10.3 从"写代码的人"到"管 agent 的人"

最后，说清楚这个转变。

以前。 你是执行者。需求进来，你写代码。工具是你的手。

现在。 你是管理者。需求进来，你拆任务、选 agent、定验收。工具是你的团队。

我用了几个月，才彻底转过这个弯。

这个转变有几个具体动作。

第一，任务比代码重要。 给 agent 的任务描述，决定输出质量。第三章 里两个 agent 的活儿干得漂亮，因为任务写得清楚：做什么、测试怎么跑、汇报什么。写任务，比写代码更值得花时间。

第二，审查比生成重要。 agent 写代码，你审代码。第三章 的合并冲突，第六章 的独立审查，都是这个道理。你的价值在"判断"不在"打字"。

第三，流程比工具重要。 装 Paseo 一天学会，配好 workflow 才叫会用。schedules 定时、profiles 分工、skills 交接，这些是把工具变成流程的东西。

第四，信任但验证。 agent 说测试过了，你要看日志。agent 说改完了，你要看 diff。我用 Paseo 跑下来的原则：让 agent 自己证明，你负责抽检。这个原则，我写进了每次任务的验收标准。

> **行动清单**：
>
> - ☐ 装好 Paseo，跑通一个真实 agent 任务
> - ☐ 用 worktree 并行跑两个任务，体验隔离
> - ☐ 建一个 schedule，让 agent 定时干活
> - ☐ 配好 2-3 个 agent profiles，按任务分工
> - ☐ 组一次 Claude 规划 + Codex 实现 + 独立审查的流程

10.4 最后一句话

Paseo 的一年，从 0 到 2 万 star。一个人，201 个版本。

这不是"AI 帮你写代码"的故事。这是"一个人管理一群 agent"的故事。

工具会变，模型会换。但这个方向不会变：你的机器上，永远有一个在工作 agent。你负责决定它干什么。我是这么安排的，你也可以。

装好它，跑起来。你的第一个并行 agent 项目，现在就能开始。



附录

>_ 附录 A：CLI 命令速查表

基于本机 Paseo CLI v0.11.2 实测。

全局选项

选项	作用		
`--host <host>`	指定 daemon 端点（host:port / TCP / socket / SSH / 配对 URL）		
`--home <path>`	选择 daemon home（默认 ~/.paseo）		
`-o, --format	json	yaml>`	输出格式
`--json`	JSON 输出		
`-q, --quiet`	只输出 ID		

agent 操作

命令	作用
`paseo run <prompt>`	创建并启动 agent
`paseo ls`	列出 agent（-a 含归档，-g 跨所有 workspace）
`paseo attach <id>`	实时流式查看输出
`paseo send <id> "..."`	给已有 agent 发后续任务
`paseo logs <id>`	查看 agent 时间线（-f 流式，--tail N）
`paseo stop <id>`	中断运行中的 agent
`paseo delete <id>`	删除 agent
`paseo archive <id>`	归档 agent（软删除）
`paseo inspect <id>`	查看 agent 详细信息（模型、模式）
`paseo wait <id>`	等待 agent 变空闲
`paseo import <id>`	导入已有 provider 会话

run 的常用参数

参数	作用
`--provider codex/gpt-5.5`	provider 或 provider/model
`--model <model>`	指定模型
`--mode	default	bypass>`	权限模式
`--new-workspace	worktree>`	开新工作区	
`--worktree-mode	checkout-branch	checkout-pr>`	worktree 模式
`--new-branch <name>` / `--base <ref>`	branch-off 参数
`--pr-number <n>`	checkout-pr 参数
`--workspace <id>`	在已有工作区运行
`--cwd <path>`	工作目录
`--env <key=value>`	设置环境变量
`--label <key=value>`	加标签
`--wait-timeout <30s	5m	1h>`	等待上限
`--output-schema <schema>`	结构化输出
`--no-wait`	启动后立即返回（后台跑）

workspace 操作

```
paseo workspace create --isolation local --path ~/dev/my-app --title main
paseo workspace create --isolation worktree --path ~/dev/my-app --base origin/main
paseo workspace ls
paseo workspace rename <id> "name"
paseo workspace archive <id>
paseo project create / ls / rename / delete
paseo clone <repo>   # 克隆 GitHub 仓库并注册为 workspace
```

provider 操作

```
paseo provider ls                    # 可用 provider 和状态
paseo provider models <provider>     # 某 provider 的模型列表
paseo provider diagnostic <provider> # 诊断安装和环境
```

定时任务

```
paseo schedule create --every 30m --name x --provider codex/gpt-5.5 --cwd ~/dev "prompt
paseo schedule create --cron "0 9 * * 1-5" --timezone Asia/Shanghai --name x ...
paseo schedule ls / inspect <id> / logs <id>
paseo schedule pause <id> / resume <id> / run-once <id> / delete <id>
```

权限与模式

```
paseo permit ls                    # 待审批列表
paseo permit allow <agent-id>      # 批准
paseo permit deny <agent-id>       # 拒绝
paseo agent mode <id> --list       # 可用模式
paseo agent mode <id> bypass       # 切到 bypass
paseo agent detach <id>            # 分离 subagent
```

daemon 管理

```
paseo daemon start / stop / restart / status / pair
paseo daemon config get daemon.listen
paseo daemon config set daemon.listen 127.0.0.1:6800
paseo reload                       # 重载 config.json（不重启）
paseo daemon set-password          # 设置密码
```

插件与终端

```
paseo plugin add owner/slug                    # 从注册表安装
paseo plugin add github:owner/repo             # 从 GitHub 安装
paseo plugin init <dir>                        # 新建插件
paseo plugin ls / reload <id> / logs <id> / remove <id>
paseo terminal create --workspace <id> --name Development
paseo terminal send-keys <term-id> "echo hi"
paseo terminal capture <term-id>
paseo script ls / start web / stop web
```

连接与远程

```
paseo --host ssh://user@host ls -a                    # SSH 远程
paseo --host workstation.local:6767 run "..."         # 远程 daemon
paseo --host 'https://app.paseo.sh/#offer=...' ls     # 配对链接
paseo pair                                            # 打印配对二维码
paseo status                                          # 看 daemon 状态
```

附录 B：常见问题排查

来源：官方文档 /docs/troubleshooting 与我的实测。

B.1 provider 找不到

症状。 `paseo provider ls` 里显示 unavailable。

原因。 Paseo 启动的是你已安装的 agent CLI，它不内置。找不到，多半是 CLI 没装、没登录、或不在 PATH。

排查。 诊断命令：

```
paseo provider diagnostic claude
```

看三行关键信息：Resolved path（解析到的路径）、Daemon PATH、Version。

修复。 确认命令在正常登录 shell 的 PATH 里，重启 Paseo。还不行，固定路径：

```
{
  "agents": {
    "providers": {
      "claude": {
        "command": ["/absolute/path/to/claude"]
      }
    }
  }
}
```

> ⚠ **注意**：`which -a claude` 找真实路径。
>
> `type -a claude` 看它是不是别名或函数。别名和函数不能用，Paseo 直接跑二进制。

B.2 为什么 Paseo 环境和终端不一样

症状。 命令在终端能跑，Paseo 里报 not found。

原因。 桌面应用从 Dock 打开时得到精简环境。Paseo 启动时跑一次登录 shell（`$SHELL -i -l -c`）捕获环境。版本管理器（asdf、mise、nvm）是常见罪魁。

经验法则。 全新终端能跑的命令，Paseo 应该也能跑。

排查。 桌面日志找 `[login-shell-env]`。看到 `failed; keeping inherited env`，说明登录 shell 有问题。慢或报错的 .zshrc 是常见原因。

B.3 改 config.json 没效果

原因。 有些设置要重启，有些不用。

修复。 先 `paseo reload`。它会应用运行时安全的设置，并告诉你哪些路径需要重启。

```
paseo reload
paseo daemon restart
```

注意。 reload 从不隐式重启。`daemon.listen` 这类启动期设置，必须 restart。重启时，运行中的 agent 继续跑，客户端自动重连。

B.4 UI 加载了但连不上

症状。 浏览器打开 Web UI，页面渲染了，但一直连不上。

原因。 反向代理没转发 WebSocket 升级。

修复。 Nginx 配置加:

```
map $http_upgrade $connection_upgrade {
  default upgrade;
  ''      close;
}

proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection $connection_upgrade;
```

Caddy 一行搞定：

```
paseo.example.com {
  reverse_proxy 127.0.0.1:6767
}
```

B.5 403 Host not allowed

症状。 通过域名访问报 403。

原因。 Host 白名单没配。Paseo 用 `daemon.hostnames` 防 DNS rebinding。

修复。 Docker 里配 `PASEO_HOSTNAMES`：

```
PASEO_HOSTNAMES: "paseo.example.com,.lan"
```

B.6 其他速查

问题	处理
端口占用	看 `~/.paseo/daemon.log`
daemon 起不来	`paseo daemon run` 前台跑，看报错
任务很贵	换便宜模型，用小任务
AppImage 打不开（Linux）	`chmod +x`；还不行 `--appimage-extract-and-run`
SSH 连接被拒	远程先 `paseo daemon status`。SSH 传输不启动 daemon
密码忘了	`paseo daemon set-password` 重置
HTTPS 下 Mixed content	反向代理传 `X-Forwarded-Proto`，配 trustedProxies

附录 C：版本时间线

数据来源：GitHub API 与官方文档，截至 2026-10-11。

关键节点

时间	事件
2025-10-13	仓库创建，秘密开发
2026-02-11	首个公开版本 v0.1.2
2026-02-17	v0.1.8，桌面版发布节奏启动
2026-04	单月 46 个版本，迭代峰值
2026-06 ~ 08	每月 23-24 个版本，稳定高速
2026-10-07	v0.11.0：插件注册表上线；新增 Muse Code、Antigravity provider
2026-10-07	v0.11.1：支持 Claude Haiku 5.5
2026-10-09	v0.11.2（当前最新）

发布数量

2026 年 2 月到 10 月，共 201 个发布版本。

月份	版本数
2026-02	13
2026-03	19
2026-04	46
2026-05	30
2026-06	24
2026-07	24
2026-08	23
2026-09	13
2026-10	9（截至 10 月 11 日）

版本命名

SemVer + 预发布标签。

稳定版 `v0.1.90`。同一版本的 beta：`v0.1.90-beta.1`。最终稳定标签是 `v0.1.92`，不是 `v0.1.93`。beta 是通往同一个稳定版的检查点。

发布渠道

渠道	说明
Stable（默认）	桌面版 36 小时内逐步推送
Beta	预发布即达，无延迟。Settings → About 切换
应用商店	经审核，可能滞后于桌面版

插件版本里程碑

版本	意义
0.8.0	插件 API 第一个破坏性发布
0.11.0	插件名/图标/截图；usage sources；插件注册表

更新建议

一个实际建议：Paseo 迭代太快，月更 20 版。跟着 stable 走，出问题切回上一个版本。工具在快速演进，书里写的命令，可能半年后就有变化。以官方文档为准。

阅读指南

时间	章节	目标
Day 1	第一章 - 第三章	从零到第一次跑通并行 agent
Day 2-3	第四章 - 第六章	掌握工作区、worktree、编排
Day 4-5	第七章 - 第十章	跨设备、自动化、扩展、心态
Day 6	附录	速查与排查